

Índice

1	Entorno y Configuración (macOS / Linux)	1
1.1	Instalación de Utilidades GNU en Mac	1
1.2	Alias de Compilación (~/.zshrc)	1
1.3	Script de Verificación de Tipografía (hash.sh)	1
1.4	Configuración Básica de Editor (~/.vimrc)	1
1.5	Remapear Caps Lock de Forma Nativa en macOS	1
1.6	Validar Binario Activo	1
2	Teoría de Juegos Imparciales y Teorema de Sprague-Grundy . . .	1
2.1	Conceptos Fundamentales de Posiciones	1
2.2	El Teorema de Sprague-Grundy y la función mex	2
2.3	Implementación Eficiente de Grundy Values con DP	2

1. Entorno y Configuración (macOS / Linux)

1.1. Instalación de Utilidades GNU en Mac

Para utilizar la suite completa de GNU (como la cabecera universal <bits/stdc++.h>) e igualar con precisión el entorno de evaluación del concurso, instala la última versión de GCC y las utilidades de núcleo de GNU mediante Homebrew:

```
|| brew install gcc coreutils
```

1.2. Alias de Compilación (~/.zshrc)

Agrega este alias al final de tu archivo de configuración de la terminal para compilar localmente de forma rápida.

Nota Crítica de Arquitectura (Apple Silicon): En procesadores M1/M2/M3 bajo macOS, la suite libsanitizer dentro de GCC para ARM64 no cuenta con soporte nativo completo en Homebrew, lo que arroja el error ld: library ‘asan’ not found. Se han removido las banderas -fsanitize=address,undefined, pero se conservan los diagnósticos estrictos de tipos, desbordamientos lógicos y conversiones implícitas peligrosas:

```
|| # Ajustar g++-15 a la version exacta instalada en tu equipo (ej. g++-14, g++-15)
|| alias c='g++-15 -Wall -Wconversion -Wfatal-errors -g -std=c++17'
```

Actualiza tu sesión corriendo en la terminal: source ~/.zshrc.

También puedes simplemente agregarlo al final como: echo .alias c='g++-15 -Wall -Wconversion -Wfatal-errors -g -std=c++17'>> ~/.zshrc

1.3. Script de Verificación de Tipografía (hash.sh)

Crea el archivo hash.sh para validar la correcta transcripción de tus plantillas de código desde el cuaderno impreso. Este script compara el hash de 6 caracteres del código limpio con el hexadecimal de KACTL.

Nota sobre Portabilidad de Hashes: Es indispensable incluir la bandera -fpreprocessed para evitar que el compilador inyecte macros internos específicos del procesador M2 o del sistema operativo macOS, lo cual corrompería el hash portable de KACTL:

```
|| #!/bin/bash
|| g++-15 -E -dD -P -fpreprocessed - | tr -d '[:space:]' | gmd5sum | cut -c-6
```

Otorga los permisos de ejecución necesarios en tu terminal:

```
|| chmod +x hash.sh
```

Úsalo de la siguiente forma para verificar fallos tipográficos: ./hash.sh <codigo.cpp

1.4. Configuración Básica de Editor (~/.vimrc)

Habilita el auto-indentado limpio para código de C++ y el mapeo de escape veloz presionando las teclas jk o kj de forma sucesiva en modo inserción:

```
|| set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul sy on
|| im jk <esc>
|| im kj <esc>
```

1.5. Remapear Caps Lock de Forma Nativa en macOS

Para evitar comandos de Linux incompatibles en macOS (como xmodmap), puedes cambiar el mapeo directamente desde la interfaz del sistema operativo:

- 1. Ve a **Ajustes del Sistema** → **Teclado**.
- 2. Haz clic en **Atajos de teclado...** y luego selecciona **Teclas de modificación**.
- 3. Cambia la acción de la tecla **Bloqueo de mayúsculas** (*Caps Lock*) a **Control** o **Escape** para una navegación ultra veloz en Vim.

1.6. Validar Binario Activo

Si Homebrew actualiza o instala una versión superior de GCC, identifica con exactitud el sufijo del comando y las rutas ejecutando:

```
|| brew info gcc
|| ls /opt/homebrew/bin/g++-*
```

2. Teoría de Juegos Imparciales y Teorema de Sprague-Grundy

2.1. Conceptos Fundamentales de Posiciones

En la convención de juego normal (el último en mover gana), definimos de forma recursiva los estados del juego:

- **Estados Terminales:** Son posiciones P (puesto que el jugador actual no puede realizar movimientos y ha perdido).
- **Posición N (Next):** Es cualquier estado desde el cual existe **al menos un movimiento** hacia una posición P . (El jugador actual se mueve a P y le hereda la muerte al rival).
- **Posición P (Previous):** Es un estado desde el cual **todos los movimientos posibles** conducen obligatoriamente a posiciones N . (Hagas lo que hagas, dejas al rival en un estado ganador).

Estrategia ICPC: Si el valor devuelto por `computeGrundy(N)` es 0, el estado actual es una P -position (pierdes). Si es > 0 , es una N -position (ganas). Si el juego consta de múltiples sub-juegos independientes concurrentes, la respuesta global es el XOR de los valores individuales de Grundy de cada sub-juego: $G_{total} = g(G_1) \oplus g(G_2) \oplus \dots \oplus g(G_k)$.

2.2. El Teorema de Sprague-Grundy y la función mex

El teorema dicta que cualquier juego imparcial bajo juego normal es equivalente a una pila de Nim. El tamaño de dicha pila ficticia para un estado G es su valor de Grundy, denotado como $g(G)$, y se calcula mediante la función del mínimo excluido (**mex**):

$$g(G) = \text{mex}(\{g(V) : G \rightarrow V\})$$

Donde $G \rightarrow V$ representa un movimiento válido desde el estado G hacia el estado V . El **mex** de un conjunto de enteros no negativos es el entero no negativo más pequeño que **no** pertenece a dicho conjunto (ej. $\text{mex}(\{0, 1, 3\}) = 2$, $\text{mex}(\{1, 2\}) = 0$).

2.3. Implementación Eficiente de Grundy Values con DP

Para calcular los valores de Grundy en juegos abstractos (como juegos de sustracción de fichas o movimientos en grafos acíclicos dirigidos - DAGs), mapeamos los estados usando Programación Dinámica y calculamos el **mex** optimizado con vectores booleanos locales o conjuntos:

```

1 // Calcula el Grundy Value para un conjunto de estados sucesores directos
2 int getMex(const vector<int>& transitions) {
3     vector<bool> present(transitions.size() + 1, false);
4     for (int val : transitions) {
5         if (val < (int)present.size()) {
6             present[val] = true;
7         }
8     }
9     int mex = 0;
10    while (present[mex]) mex++;
11    return mex;
12 }
13
14 // Ejemplo de DP para un juego de sustraccion de fichas
15 // Se pueden retirar {1, 3, 4} fichas por turno.
16 int computeGrundy(int n) {
17     vector<int> memo(n + 1, 0);
18     vector<int> moves = {1, 3, 4}; // Movimientos permitidos
19
20     for (int i = 1; i <= n; ++i) {
21         vector<int> next_states;
22         for (int m : moves) {
23             if (i - m >= 0) {
24                 next_states.push_back(memo[i - m]);
25             }
26         }
27         memo[i] = getMex(next_states);
28     }
29     return memo[n];
30 }

```